

树 → 树干：怎么「提前看出来」一个 DP 能被压成一条链

回应你在 Recitation 16 第一个问题（Max Subarray Sum / Kadane）上的追问——「像这样压缩递推链的性质，怎么判别出来？」本答复基于你已学的 Lecture 16/17 讲义术语 + 官方框架 + 推理，不引入任何你没学过的新概念。

0. 你的直觉，精确命中了官方术语（先把这件事钉死）

你连续用了三个说法，每一个都对应讲义里一个已有的概念。你缺的不是新知识，而是把这几个概念连成一张图的那根线：

你的说法	讲义官方术语	出处
「局部暴力」	locally brute-force（局部暴力枚举） —— Relate 步骤对「未知答案」的所有候选全试一遍	Lecture 16, SRT BOT 第 2 步
「树的分支 / 发散」	Branching（分支数） = 每个子问题依赖的子问题个数	Lecture 18 「Main Features of Dynamic Programs」
「剪成树干」	Branching 从 $\Theta(n)$ 压到 $\Theta(1)$	Lecture 17（Floyd–Warshall 「只剩常数个分支」）
「减少因果链」	无后效性 / 状态压缩 ——一旦知道 $x(k - 1)$ ，更早的历史全可丢弃	隐含于 SRT BOT 的 Step 1

也就是说，你问的问题在官方讲义里早就被命名了：它叫 **Branching 的大小**。判别一个 DP 能不能从树变成链，就是判别 **Branching 是 $\Theta(1)$ 还是 $\Theta(n)$** 。下面把这根线抽出来。

1. 总纲：判别「树 vs 链」= 判别 Branching，而 Branching 在 Step 1 就锁死了

DP 的复杂度由两维相乘决定（Lecture 18 的官方框架）：

$$\text{总时间} = \underbrace{\#\text{subproblems}}_{\text{状态数}} \times \underbrace{\text{Branching}}_{\text{每个状态的候选集大小}}$$

- 状态数 = 你定义了多少个子问题（一维 $O(n)$ 、二维 $O(n^2)$）。

- **Branching (分支数)** = 求解一个子问题时，要枚举多少个「更小的子问题」作候选——也就是你说的「局部暴力」的规模。

「树」还是「链」，只看第二维 **Branching**：

- $\text{Branching} = \Theta(1)$ (常数个候选，如 2 个、3 个) → 依赖图是**链或网格** (树干)。
- $\text{Branching} = \Theta(n)$ (要扫一整排前驱) → 依赖图是**树** (发散)。

Kadane 的 $\max(A[k], A[k] + x(k-1))$ 里，那个 $\max$ 就是 $\text{Branching} = 2$ 的局部暴力。它不是「没有暴力」，是暴力被压到了常数——这是 DP 的极限形态，所以你也才觉得它「不像 DP」。

2. 判别准则 (可操作的三步决策树)

第 0 步：写出「最优解的最后一个决策」。这是 DP 建模的标准动作 (last-step 分析)。问自己：最优解的最后一个元素 / 最后一步操作，它的「前驱」是谁？

第 1 步：数这个决策的候选选项个数 —— 这就是 **Branching**。

第 2 步：判断候选集的规模，根子在「约束的接续局部性」上：

[note] 判据一 (最直接) 连续约束 (contiguous) → 前驱唯一 → $\text{Branching } \Theta(1)$ → 链。
可跳约束 (subsequence) → 前驱是一个集合 → $\text{Branching } \Theta(n)$ → 树。

- **Kadane (最大子数组和)**：子数组必须 contiguous (连续)，以 $A[k]$ 结尾的子数组只能接 $A[k-1]$ ，不可能跳过 $A[k-1]$ 接 $A[k-2]$ 。所以「接续点」唯一，候选只有「单独取 $A[k]$ 」和「接上 $x(k-1)$ 」两项 → $\text{Branching } \Theta(1)$ → 链。
- **LIS (最长递增子序列)**：子序列允许隔空取物， $x(i)$ 可以接在任意满足 $A[j] < A[i]$ 的 j 后面，候选集随 i 线性膨胀 → $\text{Branching } \Theta(n)$ → 树。

[note] 判据二 (信息视角，最深刻) 问：为了决定当前元素去留，我需要记住前面元素的多少信息？

- 只需要「紧邻前驱的最优值」这一个**标量** → 无后效性成立 → 链。
- 需要「所有满足条件的更小元素」这一个**集合** → 历史不可压缩 → 树。

这正是你说的「减少因果链」的形式化：Kadane 之所以能压成一条链，是因为状态 $x(k-1)$ 已经把「从任意起点出发、到达 $k-1$ 的所有子数组」**合并成了一个标量**——未来只需要这个标量，不需要知道它是从哪个起点来的。所有更早的历史被这个标量一刀切断。这就是 **无后效性 (Markov 性质)**：过去只需影响「当前状态的取值」，而不需影响「未来的决策方式」。

3. 核心窗户纸：树 → 链不是「剪枝」剪出来的，是「子问题定

义」一步就决定的

你说「剪枝就是把树的枝子全部剪掉，变成了一个树干」。这句话抓到了现象，但**错位了发生时机**——这是最值得纠正的一点：

Kadane 不是把一棵树「剪」成了链。它定义的子问题 $x(k)$ = 「以 $A[k]$ 结尾的最大子数组和」从一开始就让递归展开只有一条链。树 $\rightarrow$ 链发生在 SRT BOT 的 Step 1 (Subproblem 定义)，不在任何「剪枝 / 合并」阶段。

对照你学过的 Lecture 16 里 LIS 第一次尝试失败的教训，这个道理完全同构：

状态定义	能否写出递推	Branching	依赖图形状
$x(k)$ = 「前 k 个元素的最大子数组和」(不带条件)	✗ 新元素 $A[k]$ 可能根本不在最优解里，信息断了	$\Theta(n)$ (要扫所有起点)	树
$x(k)$ = 「以 $A[k]$ 结尾的最大子数组和」(加约束)	✓ 接续点唯一，递推自现	$\Theta(1)$	链

第一行的状态就是 Lecture 16 里 Demaine 说的那个「非常奇怪的递推」的翻版：朴素前缀子问题信息不够，加一个「必须结束于 $A[k]$ 」的约束后，递推自己就长出来了。这正是讲义结尾那句口诀的现场：

 **Demaine**: "If you find yourself lacking information to check the desired conditions of the problem, or lack the natural subproblem to recurse on, try subproblem constraint/expansion!" (翻译：如果你发现自己缺少信息来检查问题所需的条件，或者缺少可递归的自然子问题，那就试试子问题约束/扩展！)

所以判别「树 vs 链」的终极动作，就是审问你选的状态定义：它是否强到让「最后一个决策」只有常数个选项。约束越强（锚点钉得越死），候选越少，链越粗。Kadane 把「必须结束于 $A[k]$ 」这个约束钉到极致，接续点退化成一个，于是 $\Theta(n)$ 的树塌成了 $\Theta(1)$ 的链。

4. 回到你的例子：Kadane vs LIS 一张表看清

维度	Kadane (最大子数组和)	LIS (最长递增子序列)
约束形状	contiguous (连续)	subsequence (可跳)
最后一个决策	「接 $x(k-1)$ 还是单独取」	「接在哪个 j 后面」
候选选项数	2	$\Theta(n)$
Branching	$\Theta(1)$	$\Theta(n)$
依赖 DAG 形状	链 (树干)	树
朴素复杂度	$O(n)$	$O(n^2)$

这张表就是你要的「判别模板」：两个问题同是「以某位置结尾」的条件子问题，唯一区别是「连续 vs 可跳」，这一字之差决定了 Branching 是常数还是线性，进而决定了依赖图是链还是树。你把「连续 vs 可跳」当成判别一个序列 DP 是否可压缩的第一问，就抓住了全部。

[note] Note (译者注)：你之前问过「LIS 为什么不能也压成链」——它确实不能压成 $\Theta(1)$ 分支（因为「递增」约束要求记住「所有比当前小的元素」这个集合，不是标量），但能换一把尺子压成 $\Theta(\log n)$ ：用单调的 `tails` 数组记「长度为 k 的递增子序列的最小末尾」，把「扫 $O(n)$ 个前驱」替换成「二分一次」。这给出你已有的完整复杂度阶梯： $O(n^2)$ (LIS 朴素, Branching 线性) $\rightarrow O(n \log n)$ (换度量 + 单调性) $\rightarrow O(n)$ (Kadane, Branching 常数)。三个阶梯分别对应三种「压缩」手段：不压、换度量压、约束压到唯一。这正回答了你「压缩的性质取决于什么」——取决于问题结构给了你多大的压缩空间：连续的约束空间最大（压到唯一），递增的约束次之（压到二分），任意约束最小（只能朴素）。

5. 元答案：怎么「练出」这种判别能力

「看出来」不是天赋，是三个可以刻意练习的动作：

1. 每遇到新问题，先画依赖 DAG，数入度。子问题之间的箭头，每个节点有几个箭头指进来，就是 Branching。入度 1 $\rightarrow$ 链；入度 $O(n)$ $\rightarrow$ 树。这是最诚实的判别，永远不骗你。
2. 先写「最后一个决策」，再数选项，最后问「选项能不能被一个更强的状态吸收」。三步走：候选集多大？ $\rightarrow$ 是常数还是线性？ $\rightarrow$ 换状态定义能否把它压小？第三问是 DP 设计的灵魂——把「扫候选」换成「状态里多存一个值」（LIS 的 `tails` ）、「缺什么状态补什么坐标」（Piano Fingering 的 $x(i, f)$ ）。
3. 背下几个原型，新问题先对号入座：
 - 连续（子数组/子串） $\rightarrow$ 链，Branching $\Theta(1)$ (Kadane)
 - 逐字符（前缀对） $\rightarrow$ 网格，Branching $\Theta(1)$ (编辑距离、LCS)

- 可跳（子序列）→ 树，Branching $\Theta(n)$ （LIS 朴素）
- 区间合并 → 区间 DP，Branching $\Theta(n)$ 但状态二维（矩阵链乘）
- 选/不选 → 背包网格，Branching $\Theta(1)$ 但状态二维

先对号入座，再看「有没有单调结构可挖」——这是从「会做」到「会看」的那道坎，你已经踩在门槛上了。

6. 一句话收束

判别一个 DP 是树还是链，就一件事：写出「最优解的最后一个决策」，数它的候选选项个数。选项数是常数（前驱唯一）→ 链；是 $O(n)$ （前驱是一个集合）→ 树。而这个数的大小，由「约束的连续度」决定——连续给常数，可跳给线性。Kadane 不是消除了局部暴力，而是把局部暴力从 $\Theta(n)$ 压到了 2 个选项；它也不是「剪」出来的链，而是「以 $A[k]$ 结尾」这个子问题定义，让树从一开始就没长出来。